BetterCode: Reusable

Don’t repeat yourself

Overview

- *Recap day 1* - readable
  - Help with code comprehension
  - Clean organization, proper naming & comments
- **Reusable**: Functions
  - Help with abstract thinking
  - Activity 1: Decompose a function
  - Activity 2: Reuse the function in a notebook

Why should we reduce repetition in (scientific) writing?

Examples:

- Abbreviation
- "*as previously describe [ref]*"
- "The experiment followed the same protocol as …"

Benefits:

- Avoid plagiarism
- Conciseness
- Increase readability
- Reduce cognitive load

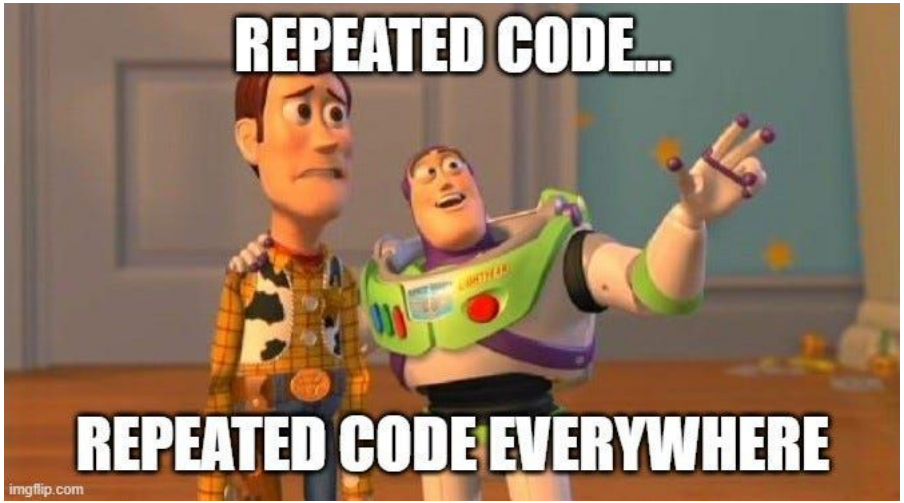
Why should code be reusable / referrable?

Increase readability

Local reasoning

Maintainable: single source of truth

Abstraction → Big-picture thinking



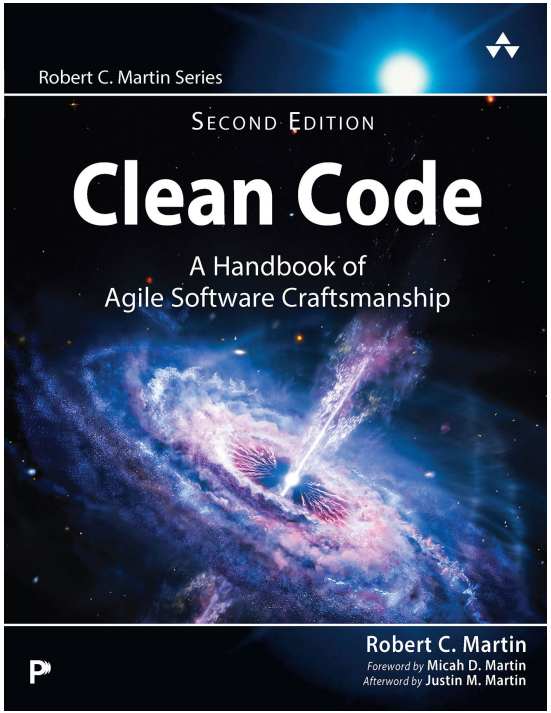
Example: day 2, activity 1 starter

What make a code reusable? Functions

- Small
- Smaller!
- Do One Thing
- Portable
- Readable
- …

Good reference: Clean Code by Rober Martin

(Chapter 3)



Demo 1: Refactor Function

find_de_genes

- Log transform the data
- Reshape to long format
- Infer sample info (control/treatment) from file name
- Perform t-test per gene & keep p-values
- Adjust p-values
- Compute group means and differences
- Compile results & filter genes

Which lines belong to the same job?

Demo 1: Refactor Function

find_de_genes

- Preprocess the data
  - Log transform
  - Reshape to long format
  - Infer sample info (control/treatment) from file name
- Compute statistic summaries
  - Perform t-test per gene
  - Keep p-values & group means
  - Calculate difference
- Adjust p-values
- Filter genes

Demo 1: Refactor Functions

Suggestion:

- Use verb to name
- Keep function within 10-30 line
  - Small
  - Do One Thing
- Different comment styles between helper and main functions
- Be explicit about assumptions

Activity 1 - Break code into smaller functions

You have 3 gene expression datasets, corresponding to 3 tissues. The script repeats the same workflow for each tissue:

1. calculate the correlation between the gene *LIF* and other genes.
2. keep genes with correlation greater than 0.5 in all 3 tissues
3. plot the expression of *LIF* and the kept genes for each tissue

Refactor the code using functions.

Activity 1 - Break code into smaller functions

1. In bullet points, what happens in the current scripts
2. Which steps group together?
3. Write helper functions: one group = one helper function
4. What should we not hardcoded?

- For each csv
  - Identify gene
  - Run correlation, pull out gene information
  - Save dataframe with correlation coefficient c
- Construct one big dataframe for all tissues
- Make plot

Activity 1 - Discussion

What repetition did you decide to abstract?

With more tissues, will this code still be generalizable?

What potential changes may require a new / modified function?

Break until 2.30pm

Demo 2: Parameterize notebooks

Do you prefer
scripts or notebooks?

```r
find_de_genes <- function(expression_matrix, alpha = 0.05) {

  expression_long <- prepare_expression_data(expression_matrix)

  # Apply the per-gene t-test function and combine the results into a single data frame.
  gene_stats <- lapply(
    split(expression_long, expression_long$gene),
    compute_single_gene_stats
  ) %>%
    bind_rows(.id = "gene")

  gene_stats$padj <- p.adjust(gene_stats$pval, method = "BH")

  significant_genes <- gene_stats %>%
    filter(padj < alpha) %>%
    arrange(pval)

  return(significant_genes)
}
```

Run The Analysis And View Results

The notebook wrapper should call the reusable function and keep the workflow readable. Describe

Perform Statistical Testing To Identify Significant Genes

```r
df <- read.csv(expression_file, row.names = 1)
significant_de_genes <- find_de_genes(df, alpha = alpha)
```

```r
cat("Number of significant genes:", nrow(significant_de_genes), "\n")
```

```
Number of significant genes: 11
```

```r
cat("Top significant genes:\n")
```

```
Top significant genes:
```

```r
print(head(significant_de_genes))
```

```
    gene         pval  Control Treatment        fc         padj
1 Gene_9 1.692844e-06 5.635662  6.172900 0.5372377 0.0001692844
2 Gene_6 3.434371e-06 5.588058  6.069368 0.4813103 0.0001717185
3 Gene_2 9.928370e-06 5.658850  6.167062 0.5082126 0.0003309457
4 Gene_7 4.143783e-05 5.635703  6.031629 0.3959257 0.0010359458
5 Gene_8 7.281749e-05 5.615413  6.192122 0.5767094 0.0014563498
6 Gene_4 1.356432e-04 5.666539  6.163368 0.4968294 0.0022607203
```

Demo 2: Parameterize notebooks

Suggestions:

- Portable (short separate sections / notebooks)
- Keep parameters on top
- Yaml config files
- One template, archive report

Activity 2 - Reusable notebook templates

Reminder from activity 1

1. calculate the correlation between the gene *LIF* and other genes.
2. keep genes with correlation greater than 0.5 in all 3 tissues
3. plot the expression of *LIF* and the kept genes for each tissue

Use the same dataset & workflow, explore the results with different parameters.

Suggestions: correlation thresholds (0.3, 0.6), correlation type (Spearman), different Gene (Gene_02, Gene_03).

Activity 2 - Discussion

- When do you keep a separate config.yaml files?
- How do you keep records of your results?

Take-away message

- Reusability ensures each idea is defined once and used consistently, making code easier to maintain, extend, and trust.